1. Kultur, samarbete & ansvar

Vi har gemensamma former för kommunikation, planering, beslut och hantering av meningsskiljaktigheter?
Vi använder Discord för kommunikation. Planeringen kunde ha varit bättre, men beslut och problem har lösts bra tillsammans.

Arbete och hinder är synliga; alla bidrar och delar ansvar för utveckling, testning och säkerhet?
Alla hjälper till med utveckling, testning och säkerhet och tar ansvar när problem uppstår.

Vi delar kunskap genom exempelvis parprogrammering och code reviews så att alla förstår projektets helhet?
Vi har haft begränsad möjlighet till parprogrammering, men code reviews har hjälpt oss att dela kunskap.

2. Produktidé, krav & feature slicing

Produktidé, målgrupp och prioriterade funktioner är tydliga och förankrade med läraren?
Vi har utgått från kursmålen när vi prioriterat funktionerna, men kontakten med läraren kunde ha varit bättre.

Vi utvecklar vertikala feature slices utifrån user flows, acceptanskriterier och BDD-scenarier?
Vi planerade våra flows, BDD-scenarier och feature slices innan utvecklingen började.

Vi har en gemensam definition av när en feature är klar?
Vi har en tydlig bild av slutresultatet, men våra krav för testning och merge kunde vara tydligare.

3. Den sociala applikationen

Användare kan skapa konto, logga in och dela innehåll eller meddelanden i ”circles”?
Ja, användare kan skapa konto, logga in och dela quiz i circles. Chattfunktion saknas.

Medlemskap, synlighet och rättigheter följer tydliga regler?
Vi kontrollerar behörigheter för både användare och circles.

Prioriterade användarflöden fungerar och fel hanteras utan informationsläckage?
De viktigaste flödena fungerar bra, men vi behöver förbättra kontrollen av informationsläckage.

4. Arkitektur, kod & ansvarsflöden

Arkitektur och kod har tydliga ansvarsgränser?
Frontend och backend är separerade och funktionerna är organiserade i olika moduler.

API-kontrakt och dataflöden är begripliga?
Vi har koll på dataflödet och använder Auth0 för autentisering. Valideringen av input kan förbättras.

Vi kan förklara hur förändringar påverkar systemet?
Ja, särskilt för de delar vi arbetat mest med. Dokumentationen för installation och användning är tydlig.

5. Git, granskning & kontinuerlig integration

Vi använder feature branches, commits och pull requests?
Ja, vi arbetar med branches, tydliga commits och pull requests.

Vi granskar kod och hanterar konflikter tillsammans?
Ja, vi gör code reviews och löser konflikter gemensamt.

6. CI – automatiserad build & test

En pipeline installerar beroenden, bygger och testar automatiskt?
Ja, vi använder bland annat npm audit, SAST och unit tests. Gherkin finns, men Cucumber och Postman är inte konfigurerade.

Misslyckade kontroller hindrar merge till main?
Ja, en misslyckad kontroll stoppar merge till main.

Vi kan förklara pipelineflödet?
Ja, vi kan förklara processen och hur vi hanterar resultat och fel.

7. Kvalitet, BDD & regressioner

BDD-scenarier styr utveckling och verifiering?
Vi använder BDD, negativa tester, npm audit och SAST för att kontrollera kvalitet och säkerhet.

Vi kan visa hur tester förebygger regressioner?
Ja, våra tester hjälper oss att upptäcka om nya ändringar påverkar tidigare funktionalitet.

Vi kan motivera testnivåer och bedöma testtäckning?
Vi har använt BDD och TDD, men vi har ingen automatisk mätning av testtäckningen.

8. Shift-left & säkerhetsautomation

Vi identifierar säkerhetsrisker tidigt?
Vi använder SAST och npm audit för att hitta säkerhetsproblem. Problemen diskuteras och hanteras i gruppen.

Säkerhetskontroller är automatiserade?
SAST och npm audit körs automatiskt. Secrets lagras i en `.env`-fil som inte finns i repositoryt.

Vi följer upp säkerhetsfynd?
Ja, vi diskuterar säkerhetsproblemen och åtgärdar dem tillsammans.

9. Identitet, behörighet & integritet

Autentisering och behörighet hanteras på ett säkert sätt?
Ja, vi använder Auth0 och gör behörighetskontroller i backend.

Vi testar att användare inte får åtkomst till fel information?
Ja, vi använder både automatiska och manuella tester.

Vi begränsar personuppgifter och skyddar känslig information?
Vi sparar endast nödvändiga uppgifter, främst e-postadresser. Lösenord hash-as och sessionsdata sparas i cookies.

10. Monitorering & återkoppling

Vi kan upptäcka och felsöka driftproblem?
Vi försöker hålla loggningen till ett minimum och har därför begränsad monitorering.

Problem blir arbetsuppgifter och följs upp?
Vi använder GitHub Issues för att registrera problem och Discord för att diskutera dem.

11. Projektberättelse, överlämning & redovisning

Vi kan demonstrera en feature från BDD till fungerande funktion?
Ja, vi kan visa hela vägen från BDD-scenario till färdig och testad funktion.

Projektberättelsen beskriver teknik, samarbete, problem och lärdomar?
Vi har inte skrivit projektberättelsen ännu, men kommer att ta upp detta under presentationen.

Alla kan förklara projektet och AI-användningen?
Ja, alla kan beskriva projektet på en övergripande nivå. AI har använts som hjälp och resultaten har kontrollerats innan användning.

Repository, dokumentation och rapporter är förberedda?
Ja, dokumentationen finns i README och de individuella rapporterna förbereds.

Avslutande kontrollfråga

Kan vi visa hur ett användarbehov blir en testad och säker förändring i applikationen?
Ja, vi kan visa hur ett BDD-scenario utvecklas till en fungerande och testad funktion.